

Zero_Nine

项目全景分析文档

版本 v2.1.0 | 2026-05-17

Rust Orchestration Kernel | 9 Crate Workspace

MIT License | Manus AI

Zero_Nine — 项目全景分析文档

版本: v2.1.0 | 构建日期: 2026-05-17 语言: Rust (Edition 2021) | 9 Crate Workspace 许可证: MIT | 作者: Manus AI

目录

1. 项目定位与核心价值
2. 设计哲学
3. 总体架构: 十三层体系
4. 工作区结构与依赖关系
5. 核心数据模型
6. 四层工作流详解
7. CLI 命令体系
8. 运行时目录
9. 演化引擎
10. 治理系统
11. Subagent 调度
12. 云同步架构
13. 安全设计
14. CI/CD 流水线
15. 宿主集成
16. SDK 与桥接
17. 测试覆盖

1. 项目定位与核心价值

是什么

Zero_Nine 是一个 Rust 编排内核 (Orchestration Kernel) , 将四个上游项目的核心语义统一为一个 CLI 工具:

上游项目	提取语义	对应 Crate
OpenSpec	提案/设计/任务工件管理	zn-spec
Superpowers	结构化执行 + 验证	zn-exec
Ralph-loop	长时调度循环 + 恢复	zn-loop
OpenSpace	技能评分 + 演化候选	zn-evolve

解决什么问题

AI 辅助编程面临三个根本挑战:

- 1. 需求模糊 — 用户目标不完整, 需要结构化澄清
- 2. 执行不可靠 — 单次 AI 调用无法保证质量和正确性
- 3. 无法进化 — 每次从零开始, 没有从历史中学习

Zero_Nine 通过四层架构系统性地解决这些问题:

- Brainstorming — 将模糊目标转化为精确规格
- Spec Capture — 将规格转化为可执行计划
- Execution — DAG 调度 + 验证门控 + 失败恢复
- Evolution — 评分 + 信念追踪 + 课程学习

用户入口

```
/zero-nine <goal>
```

通过 Claude Code 或 OpenCode 的 slash 命令触发, Rust 核心接管后续所有工作。

2. 设计哲学

五大设计原则

原则	含义	实现
Steering > Control	引导而非强制	Brainstorming 回合制 + 人工审批门控
Observability First	所有操作可观测	NDJSON 事件日志 + 审计链 + Metrics
Recovery by Design	失败可恢复	状态持久化 + 断点续跑 + 重试策略
Feedback-Driven Evolution	从历史中学习	5维奖励模型 + 贝叶斯信念 + ELO课程
Plugin Architecture	可插拔	宿主适配器 + 技能资产 + MCP 服务

约束工程学双重策略

- **Harness (缰绳)**：给 AI 设定明确边界（DAG 约束、验证门控、策略引擎）
- **Environment (环境)**：塑造环境使正确行为自然发生（技能库、信念系统、课程学习）

3. 总体架构：十三层体系

Layer 1: Goal Intake	- 用户输入 /zero-nine <goal>
Layer 2: Brainstorming	- 回合制澄清, 生成需求包
Layer 3: Spec Capture	- 提案 + 设计文档 + 任务图
Layer 4: Planning	- DAG 验证 + 执行策略选择
Layer 5: Workspace Prepare	- Git Worktree 隔离准备
Layer 6: Subagent Dispatch	- 多代理分派 + 角色分配
Layer 7: Task Execution	- TDD Cycle + 实现
Layer 8: Verification	- 测试 + 代码审查 + 门控
Layer 9: Branch Management	- 完成分支 + PR 自动化
Layer 10: Scoring	- 执行评分 + 证据收集
Layer 11: Belief & Reward	- 贝叶斯更新 + 5维奖励
Layer 12: Curriculum Learning	- ELO 难度 + 课程推荐
Layer 13: Observability	- 事件流 + Metrics + 审计

Crate 与层次映射

Crate	负责层	核心职责
zn-host	Layer 1-2	宿主检测、适配器导出、GitHub 集成
zn-spec	Layer 2-3	头脑风暴、提案生成、规格验证
zn-exec	Layer 4-9	策略引擎、DAG 调度、Subagent、验证
zn-loop	Layer 1-13	编排循环、状态机、调度、恢复
zn-evolve	Layer 10-13	评分、蒸馏、信念、奖励、课程、云同步
zn-bridge	Layer 6	gRPC 桥接、MCP 服务器
zn-sdk	全部	统一 Facade，编程接口
zn-types	全部	共享类型定义（100+ 结构体/枚举）
zn-cli	入口	CLI 二进制、TUI 仪表板

4. 工作区结构与依赖关系

Crate 清单

```
Zero_Nine/
├─ Cargo.toml                                # workspace (resolver = "2", edition 2021)
├─ crates/
│   ├─ zn-types/                            # 共享类型 (1290+ lines, 42 tests)
│   ├─ zn-spec/                             # 规格与工件 (SQLite 会话搜索)
│   ├─ zn-exec/                             # 执行策略引擎 (111 tests)
│   ├─ zn-loop/                             # 编排循环 (31 tests)
│   ├─ zn-evolve/                           # 演化引擎 (64 tests)
│   ├─ zn-host/                             # 宿主适配器
│   ├─ zn-bridge/                           # gRPC 桥接 (tonic + prost)
│   ├─ zn-sdk/                              # 统一 Facade SDK
│   └─ zn-cli/                              # CLI 二进制 (zero-nine)
├─ adapters/                                # 宿主适配器模板
├─ docs/                                    # 架构文档
├─ .github/workflows/                       # CI/CD 配置
├─ Dockerfile                              # 多阶段构建
├─ justfile                                # 任务运行器
└─ demo_project/                           # 测试项目
```

依赖关系

```
zn-types (底层, 无内部依赖)
├─ zn-spec ───┐
├─ zn-host ───┐
├─ zn-bridge ─┐
├─ zn-evolve ─┐
│               └─ zn-exec ───┐
│               └──────────┐
│               │             └─ zn-loop
│               │             └─ zn-sdk
│               │             └─ zn-cli
└────────────────┘
```

关键外部依赖

类别	依赖	用途
序列化	serde, serde_json, serde_yaml	所有数据模型
异步	tokio, tokio-stream	异步运行时
gRPC	tonic, prost	桥接通信
HTTP	reqwest	AI API 调用 + 云同步
CLI	clap, rustyline	命令行 + REPL
TUI	ratatui, crossterm	终端仪表板
数据库	rusqlite	记忆搜索
测试	insta	快照测试

5. 核心数据模型

5.1 项目与提案

```
ProjectManifest
├─ version: "2.1.0"
├─ name: String
├─ default_host: HostKind (ClaudeCode | OpenCode | Terminal)
├─ policy: Policy (max_retries, verify_before_complete, auto_evolve...)
├─ github_repo: Option<GitHubRepo>
└─ bridge_address: Option<String>

Proposal
├─ id, goal, description
├─ problem_statement, scope_in, scope_out
├─ constraints, acceptance_criteria, risks
├─ dependencies, non_goals
├─ execution_strategy: ExecutionStrategy
└─ issue_sources: Vec<IssueSource>    // GitHub 问题溯源
```

5.2 任务图 (DAG)

```
TaskGraph
├─ tasks: Vec<TaskItem>
│   └─ TaskItem
│       ├─ id, title, description
│       ├─ status: TaskStatus
│       ├─ depends_on: Vec<String>
│       ├─ kind: ExecutionMode
│       ├─ contract: TaskContract
│       ├─ max_retries: u8
│       └─ preconditions: Vec<String>
├─ edges: Vec<TaskDependencyEdge>
└─ validate_dag() → DagValidationResult
    ├─ valid: bool
    ├─ errors, warnings
    ├─ critical_path: Vec<String>
    └─ max_depth: usize
```

5.3 状态机

LoopStage 状态流转:

```
Idle → SpecDrafting → Ready → RunningTask → Verifying → Archived → Completed
                                     ↓
                               Retrying (max 2)
                                     ↓
                               Escalated
```


5.4 执行与验证

```
ExecutionReport
├─ success: bool
├─ tests_passed, review_passed
├─ evidence: Vec<EvidenceRecord>
├─ governance_summary
├─ token_budget: TokenBudget
├─ drift_check: Option<DriftCheckResult>
├─ failure_classification: Option<FailureClassification>
└─ skill_evaluation: SkillEvaluation
```

5.5 治理

```
PolicyEngine
├─ rules: Vec<PolicyRule>
├─ rbac: RBACStore           // 角色权限
├─ rate_limits: HashMap
└─ audit_chain: Vec<AuditEntry> // SHA256 哈希链

AuditEntry
├─ id (UUID), timestamp, action, actor
├─ decision: PolicyDecision
├─ risk_level: ActionRiskLevel
├─ prev_hash: String (SHA256) // 防篡改
└─ entry_hash: String (SHA256)
```

5.6 演化

```
BeliefTracker
├─ beliefs: HashMap<String, BeliefState>
│   └─ BeliefState { confidence, evidence_count, trend }
├─ questions: Vec<BeliefQuestion>
└─ decisions: Vec<BeliefDecision>

RewardModel - 5 维度: code_quality, test_coverage,
               user_satisfaction, execution_speed, token_efficiency

IntegrationEngine
├─ scorer: SkillScorer
├─ distiller: SkillDistiller
├─ reward_model: RewardModel
├─ belief_tracker: BeliefTracker
├─ curriculum_manager: CurriculumManager
└─ ai_client: Option<AIClient>
```

5.7 云同步

```
VersionVector
├─ node_id: String
└─ clocks: HashMap<String, u64>    // skill_id → 逻辑时钟

CloudSyncState
├─ distilled_skills: Vec<DistilledSkill>
├─ skill_bundles: Vec<SkillBundle>
├─ skill_count: usize
├─ version_vectors: VersionVector
└─ last_synced_at: Option<DateTime<Utc>>
```

冲突解决: 比较版本向量时钟, 高者胜出; 相等时本地优先 LWW

6. 四层 workflow 详解

6.1 第一层：Brainstorming（头脑风暴）

将模糊的用户目标转化为精确的需求包。

流程：

```
用户: /zero-nine "Add authentication to the API"
↓
Rust: 生成澄清问题 (5-8 个)
↓
用户回答: "JWT, not OAuth. Support refresh tokens."
↓
回合制: 继续提问直到 Verdict = Ready
↓
输出: RequirementPacket { goal, context, constraints, acceptance_criteria }
```

特性：回合制、自动裁决、可恢复（--resume）

6.2 第二层：Spec Capture（规格捕获）

将需求包转化为完整的提案文档集合。

产出物：

```
.zero_nine/proposals/<id>/
├─ proposal.json    # 结构化提案
├─ design.md        # 设计文档
├─ tasks.md         # 任务清单
├─ dag.json         # 任务依赖图
├─ progress.json    # 进度跟踪
└─ verification.md  # 验证计划
```

验证：DAG 无环验证、规格完整性检查、约束条件检查

6.3 第三层：Execution（执行）

DAG 调度： – 任务仅当所有依赖完成才可执行 – 最多 2 个并发任务 – finish_branch 待处理时，并发限制为 1

工作空间策略： | 策略 | 说明 ||-----|-----|| GitWorktree | 每个任务独立 worktree，最多 2 槽位 || In-Place | 原地执行（无初始提交时降级） || Sandboxed | 沙箱执行（预留） |

验证门控： 1. 自动测试（cargo test / pytest / npm test） 2. 代码审查（git diff --check） 3. 策略检查（风险级别评估） 4. 人工审批（Critical 风险需要 ApprovalTicket）

失败恢复：

```
Failed → Retrying (1/2) → Retrying (2/2) → Escalated → Evolution Candidate
```

6.4 第四层：Evolution（演化）

执行后评估流程： 1. 评分：0.0–1.0，基于 5 维度加权证据 2. 蒸馏：从执行模式中提取可复用技能 3. 信念更新：贝叶斯信念追踪器调整置信度 4. 课程调整：ELO 评级系统调整任务难度 5. 候选生成：AutoFix / AutoImprove / AutoLearn

7. CLI 命令体系

核心命令（8 个）

命令	用途	关键参数
init	创建 .zero_nine/ + manifest	--project, --host
brainstorm	头脑风暴	--project, --host, --goal, --resume
run	执行完整四层工作流	--project, --host, --goal
status	显示当前状态	--project
resume	从断点继续	--project, --host, --dry_run
export	导出宿主适配器	--project
dashboard	TUI 终端仪表板	--project
bridge-server	启动 gRPC 服务	--port (默认 50051)

技能管理（14 个子命令）

create , list , view , patch , edit , delete , validate , score , scores , suggest , distill , list-distilled , match , apply

扩展命令

类别	子命令	用途
Memory	init, add, remove, read, search, recent, stats	项目记忆管理 (SQLite)
MCP	init, list, call	MCP 服务管理
Cron	schedule, remind, cancel, list, stats	定时任务
Subagent	dispatch, history, ledger	多代理分派
Governance	check, matrix, ticket, tickets, approve, reject, stats	策略与审批
Governance/audit	search, stats, verify	审计链
Governance/compliance	generate, check	合规报告
GitHub	import, create-pr, comment, summarize	GitHub 集成
Observe	events, proposal, trace, stats, metrics	可观测性
Evolve	decision, snapshot, reset, sync	演化 + 云同步

使用示例

初始化项目

```
zero-nine init --project . --host opencode
```

头脑风暴

```
zero-nine brainstorm --project . --host opencode --goal "Add user auth"
```

执行完整流程

```
zero-nine run --project . --host opencode --goal "Implement API v2"
```

启动 TUI 仪表板

```
zero-nine dashboard --project .
```

启动 gRPC 桥接

```
zero-nine bridge-server --port 50051
```

查看审计链

```
zero-nine governance audit search --action deploy
```

云同步（推送）

```
zero-nine evolve sync --project . --push --endpoint https://api.example.com --token xxx
```

8. 运行时目录

```
.zero_nine/
├─ manifest.json           # 项目配置
├─ mcp_config.yaml         # MCP 服务器配置
├─ brainstorm/            # 头脑风暴会话
│   └─ sessions/          # 会话存档
│       └─ latest-session.json # 当前活跃会话
├─ proposals/<id>/        # 提案工件
│   └─ proposal.json, design.md, tasks.md
│   └─ dag.json, progress.json, verification.md
├─ evolve/                # 演化数据
│   └─ distilled_skills.ndjson
│   └─ skill_versions.json
│   └─ cloud_sync.json
│   └─ skills/            # 技能文件
├─ memory/                # 记忆系统
│   └─ MEMORY.md, USER.md
│   └─ sessions.db        # SQLite 搜索
├─ cron/                  # 定时任务
│   └─ scheduler_state.json
├─ loop/                  # 调度状态
│   └─ session-state.json
│   └─ iteration-log.ndjson
└─ runtime/               # 运行时
    └─ events.ndjson      # 事件流（仅追加）
    └─ verification/      # 验证工件
```

设计原则：单一真相源、仅追加写入、可恢复、可选镜像

9. 演化引擎

评分系统

```
SkillEvaluation { score: 0.0-1.0 }  
5 维度: code_quality, test_coverage, user_satisfaction,  
        execution_speed, token_efficiency
```

蒸馏系统

从执行历史提取可复用技能: 1. 模式提取 → 2. 场景匹配 → 3. 置信度计算 → 4. 技能生成 (SKILL.md)

贝叶斯信念追踪

```
posterior = (likelihood × prior) / evidence  
trend = (recent_avg - historical_avg) / historical_avg
```

ELO 课程学习

```
成功: new_rating = old_rating + K × (1 - expected)  
失败: new_rating = old_rating - K × expected
```

三系统融合

```
ExecutionReport → scorer → distiller → belief_tracker  
                → reward_model → curriculum_manager  
                → IntegrationEngine.fuse() → IntegratedDecision
```

AI 客户端

可选的外部 AI API 集成 (OpenAI / Claude / Custom) : - collect_feedback() — 收集用户反馈 - generate_enhancement() — 生成改进建议 - distill_with_llm() — LLM 辅助技能蒸馏

10. 治理系统

策略引擎

```
PolicyRule { condition, action, risk_threshold, roles, exceptions }  
PolicyDecision: Allow | Ask | Deny | Escalate
```

RBAC（基于角色的访问控制）

```
GovernanceRole: Admin | Approver | Executor | Reviewer | Observer  
RolePermission { role, permissions, risk_levels }  
RBACStore { roles, users }
```

审计链（SHA256 防篡改）

```
AuditEntry { id, timestamp, action, actor, decision,  
              prev_hash (SHA256), entry_hash (SHA256) }  
  
验证: entry_hash = SHA256(id + timestamp + action + actor + prev_hash + details)
```

失败分类

```
FailureCategory: EnvironmentDrift | ToolError | VerificationFailed  
                | PolicyBlocked | HumanRejected | ResourceExhausted | Timeout | Unknown
```

合规报告

```
ComplianceReport {  
    period, total_actions, policy_violations,  
    approval_rate, escalation_rate,  
    mean_resolution_time, compliance_score  
}
```

11. Subagent 调度

多代理架构

```
SubagentDispatcher { parallel_limit (max 2), available_roles, dispatch_strategy }  
AgentRole: Planner | Executor | Reviewer | Coordinator  
执行路径: Cli | Bridge | Hybrid
```

分派流程

1. 分析任务依赖图
2. 确定可并行任务
3. 分配 Agent 角色
4. 创建 SubagentBrief
5. 通过 Cli/Bridge/Hybrid 执行
6. 收集结果并验证

恢复机制

```
SubagentRecoveryLedger {  
    runs: Vec<SubagentRecoveryRecord>,  
    max_retries: u8,  
}
```

12. 云同步架构

版本向量

```
VersionVector { node_id, clocks: HashMap<skill_id, u64> }  
操作: increment(skill_id), merge(other)
```

冲突解决

1. content_hash 相同 → 跳过
2. 不同 → 比较时钟:
 - 远程高 → remote_wins
 - 本地高 → local_wins
 - 相等 → 本地优先 LWW
3. 更新版本向量

客户端

```
CloudSyncClient {  
    config: CloudSyncConfig { endpoint_url, auth_token, node_id, auto_sync },  
    http: request::Client,  
}  
操作: upload_state(), download_state(), merge_state()
```

13. 安全设计

已修复漏洞 (v1.0.2)

#	漏洞类型	修复方案
1	API Key 明文序列化	脱敏输出 + 加密存储
2	Shell 注入	参数白名单 + 转义
3	TLS 降级攻击	强制 HTTPS + 证书验证
4	路径遍历	路径规范化 + 白名单
5	SQL 注入	rusqlite 参数化查询
6	审计链篡改	SHA256 哈希链验证
7	Token 泄露	令牌自动过期 + 范围限制
8	速率限制绕过	多级限流 + IP 追踪
9	越权访问	RBAC 强制检查
10	拒绝服务	超时控制 + 资源限制

14. CI/CD 流水线

GitHub Actions

Job	触发	内容
fmt	push/PR	cargo fmt --check
clippy	push/PR	cargo clippy -D warnings
test	push/PR	Ubuntu + macOS + Windows 矩阵测试
audit	push/PR	RustSec 安全审计
coverage	main only	Tarpaulin + Codecov
docker	push/PR	Docker 构建验证

Release 流程

```
push tag v*.*.* → build (5 平台) → release (GitHub) → publish (crates.io)
```

```
构建平台: linux x86_64, linux aarch64 (cross), macOS x86_64,  
           macOS aarch64, windows x86_64
```

Docker 多阶段构建

```
Stage 1: rust:1-slim → cargo build --release -p zn-cli
```

```
Stage 2: debian:bookworm-slim → 复制二进制, ENTRYPOINT ["zero-nine"]
```

justfile 任务运行器

```
test          → cargo test --all-targets  
fmt           → cargo fmt --check  
clippy        → cargo clippy --all-targets --all-features -- -D warnings  
review        → cargo insta review  
docker-build → docker build -t zero-nine .  
docker-run    → docker run --rm zero-nine --help  
coverage      → cargo tarpaulin --all-features --workspace --out Html
```

15. 宿主集成

Claude Code 适配器

```
adapters/claude-code/  
├─ .claude-plugin          # 插件清单  
├─ commands/  
│   └─ zero-nine.md        # 路由命令  
└─ skills/  
    └─ zero-nine-orchestrator/  
        └─ SKILL.md         # 编排技能
```

OpenCode 适配器

```
adapters/opencode/  
├─ commands/  
│   └─ zero-nine.md  
└─ skills/  
    └─ zero-nine-orchestrator/  
        └─ SKILL.md
```

宿主类型

```
HostKind: ClaudeCode | OpenCode | Terminal  
detect_host() → 自动检测可用宿主  
export_adapter_files() → 生成适配器文件
```

16. SDK 与桥接

SDK Facade

```
let zn = ZeroNine::from_project("./demo_project", HostKind::OpenCode)?;  
zn.init()?;  
zn.brainstorm("Add authentication"?);  
zn.run_goal("Implement OAuth2"?);  
zn.status()?;  
zn.export()?;
```

方法: `new()`, `host()`, `init()`, `brainstorm()`, `brainstorm_headless()`, `run_goal()`, `run_goal_headless()`, `resume()`, `resume_headless()`, `status()`, `export()`, `validate_spec()`, `brainstorm_host_turn()`, `run_dry()`, `resume_dry()`

gRPC 桥接

```
zero-nine bridge-server --port 50051
```

服务: Dispatch (任务分派)

Evidence (证据收集)

Status (状态查询)

MCP 集成: github, linear, filesystem

17. 测试覆盖

测试统计

Crate	测试数	覆盖范围
zn-types	42	数据模型、状态机、DAG 验证
zn-spec	41	技能格式、策略引擎、会话搜索
zn-exec	111	执行计划、治理、Subagent、验证
zn-loop	31	调度器、状态机、持久化
zn-evolve	64	评分、蒸馏、信念、云同步
zn-cli	11	CLI 集成、快照测试
总计	300	

快照测试 (18 个)

使用 insta 框架，覆盖： – DAG 验证结果（有效/有环） – 状态转移记录 – 提案序列化 – Manifest JSON – 任务 Markdown 渲染 – 技能渲染 – 蒸馏技能 – 审计条目 – 审批票据

质量指标

- 测试通过率: 99.7% (299/300 pass, 1 ignored)
- Clippy 警告: 0

- 格式化: 通过
- TODO 数量: 0

18. 与 Qoder CLI 能力对比及增强方向

18.1 Qoder CLI 有但 Zero_Nine 缺的能力

Agent 专业化体系

Qoder CLI 提供多种专业 Agent 类型，而 Zero_Nine 的 Subagent 目前是通用任务分派：

Qoder Agent 类型	能力	Zero_Nine 对应	差距
Explore	快速代码库搜索	无（Subagent 是通用任务分派）	大
Plan	架构设计+方案规划	仅 WritingPlans 执行模式	中
general-purpose	通用复杂任务	Subagent 分派	小
debug	系统性调试	FailureClassification 但无调试流程	大
security-review	安全审查	有策略引擎但无专门安全审计	中
skill-creator	技能创建	skill distill 但无交互引导	中

增强方向：在 Subagent 系统中引入专业化 Agent 类型：

```
enum AgentSpecialization {
    CodeExplorer,    // 快速搜索/理解代码
    Architect,       // 方案设计/架构决策
    Debugger,        // 问题诊断+根因分析
    Reviewer,        // 安全审查+代码质量
    SkillDistiller,  // 从执行历史蒸馏技能
    Tester,          // 测试生成+验证
}
```


Hook 系统（事件驱动扩展）

Qoder Hook	能力	Zero_Nine	差距
pre-tool-call	工具调用前拦截	策略引擎但非事件驱动	大
post-tool-call	工具调用后处理	事件日志但无实时处理	大
pre-message	用户消息预处理	Brainstorming 但无 Hook	中

增强方向：将 events.ndjson 从被动日志升级为事件总线，支持注册 Hook 处理器：

```
pub enum EvolutionEvent {
    TaskCompleted(ExecutionReport),
    TaskFailed(FailureClassification),
    SkillDistilled(DistilledSkill),
    ConflictDetected(MergeResult),
}

pub trait EventHook {
    fn on_event(&self, event: &EvolutionEvent) -> Result<HookAction>;
}
```

MCP Server 深度集成

维度	Qoder CLI	Zero_Nine	差距
MCP 协议	原生支持，多服务器并行	有 mcp_config.yaml 但仅是静态配置	大
工具发现	自动发现 MCP 工具	硬编码命令	需增强
动态调用	运行时动态调用外部 MCP	仅定义格式	需增强

增强方向：将 zn-bridge 的 MCP 服务从静态配置升级为动态发现+运行时调用，支持连接外部 MCP 服务器（GitHub、Linear、Jira 等）作为任务执行的工具源。

IDE 集成

Qoder 能力	说明	Zero_Nine	差距
IDE 集成	VS Code 等深度集成	仅 CLI	大
Context 压缩	对话上下文自动压缩	仅状态序列化	中

增强方向：通过 LSP 协议或 VS Code Extension 提供 IDE 插件，让用户在编辑器内直接触发 /zero-nine 命令，实时查看执行进度和结果。

Skills 系统模块化与可发现性

维度	Qoder Skills	Zero_Nine Skills	差距
格式	SKILL.md + metadata	SKILL.md（无 metadata）	中
发现	自动扫描+注册	手动注册到 evolve/skills/	中
激活	模式匹配自动激活	手动 skill apply	中
评分	无	有 0-1 评分系统	Zero_Nine 更强

增强方向：为技能文件添加元数据头部：

```
---
name: zero-nine-tdd-cycle
version: 1.2.0
description: Test-driven development cycle
triggers: ["implement", "write code", "new feature"]
confidence: 0.87
last_distilled: 2026-05-17
---
```

18.2 Zero_Nine 有但 Qoder CLI 缺的能力

演化引擎（核心差异化）

能力	Zero_Nine	Qoder CLI
贝叶斯信念追踪	有	无
5 维奖励模型	有	无
ELO 课程学习	有	无
技能蒸馏	有	无
三系统融合决策	有	无

治理系统

能力	Zero_Nine	Qoder CLI
RBAC 角色权限	有	无
SHA256 审计链	有	无
策略引擎（Allow/Deny/Escalate）	有	无
合规报告生成	有	无
审批票据系统	有	无

DAG 任务编排

能力	Zero_Nine	Qoder CLI
DAG 验证+调度	有	无
并行限制	有 (max 2)	无
依赖驱动执行	有	无

云同步

能力	Zero_Nine	Qoder CLI
版本向量冲突检测	有	无
LWW 冲突解决	有	无

18.3 增强建议优先级

高优先级：Agent 专业化 + Hook 系统

影响范围：zn-exec + zn-loop + zn-evolve

收益：让 Subagent 从"通用分派"升级为"智能专家"，支持事件驱动扩展。

中优先级：MCP 动态调用 + Skills 元数据

影响范围：zn-bridge + zn-evolve/skill_registry

收益：连接外部工具生态，技能自动发现+激活。

低优先级：IDE 集成 + Context 压缩

影响范围：新 crate zn-lsp 或 VS Code Extension

收益：提升用户体验，从 CLI 扩展到编辑器内。

18.4 架构融合建议



总结：Zero_Nine 在演化学习和治理安全方面领先 Qoder CLI，但在 Agent 专业化、MCP 生态集成、Hook 事件驱动和 IDE 集成方面可以借鉴 Qoder CLI 的成熟模式。核心差异化优势（贝叶斯信念、ELO 课程、审计链、云同步）应保持并强化，而非被替代。

19. 战略方向：多 Agent 编排基础设施

项目愿景：构建一个安全、稳定、有自我进化能力的 Agent 驾驭工程框架和环境工程框架，为多 Agent 协同合作提供安全保障，与任何 Agent 无缝连接。

19.1 核心判断

与 Qoder CLI 的差距，大部分不需要再追。Qoder CLI 是单 Agent 工具，Zero_Nine 的目标是多 Agent 编排框架——两者不在同一个赛道。追其 IDE 集成、Context 压缩等功能，会模糊核心定位。

应聚焦的方向：构建 Agent 互联的基础设施。

19.2 方向一：Agent-Agnostic Protocol (Agent 无关协议)

目标：让 Zero_Nine 能编排任何 Agent，不限于 Claude Code 或 OpenCode。

当前状态：HostKind 只有 `ClaudeCode` | `OpenCode` | `Terminal`，本质是把 CLI 命令封装成适配器。

应该做的：定义 Agent Interface Protocol (AIP)

Agent Interface Protocol (AIP)

└─ Capability Discovery

└─ Task Acceptance

└─ Evidence Reporting

└─ Health & Status

└─ Recovery Protocol

– Agent 能做什么

– Agent 接受任务的格式

– Agent 返回执行结果的格式

– Agent 存活状态

– Agent 失败后如何恢复

这不是"适配"，而是"协议"。任何 Agent 只要实现这个协议，就能被 Zero_Nine 编排。就像 HTTP 是 Web 的协议，AIP 是 Agent 编排的协议。

具体行动： – 将 zn-host 从"宿主适配器"升级为"Agent 协议网关" – 设计 AIP 的 gRPC/JSON-RPC 规范 – 提供 Agent SDK (Rust/Python/TypeScript)，让第三方 Agent 快速接入

19.3 方向二：Multi-Agent Trust Framework (多 Agent 信任框架)

目标：多个 Agent 协作时，确保安全、隔离、可审计。

当前状态：有审计链和 RBAC，但面向的是单一执行者。

应该做的：

Trust Framework

- └ Identity – 每个 Agent 有唯一身份标识
- └ Permission Boundaries – Agent A 不能访问 Agent B 的工作空间
- └ Communication Channel – Agent 间通信必须经过策略引擎
- └ Action Audit – 每个 Agent 的所有操作被独立审计
- └ Byzantine Tolerance – Agent 可能撒谎/失败/作恶，系统仍能正确运行
- └ Escalation Path – 不信任的 Agent 行为上报人工

这不只是安全，而是多 Agent 环境下的信任博弈。当 Agent 数量从 1 增长到 10、100 时，这个问题会从“有没有”变成“致命性”。

具体行动： – 为每个 Agent 分配唯一身份（Agent ID，非 user ID） – 审计链增加 agent_id 字段 – 策略引擎增加跨 Agent 规则（Agent A 不能操作 Agent B 创建的资源） – 设计“信任评分”：基于历史执行记录的动态信任度

19.4 方向三：Environment Engineering（环境工程）

目标：为 Agent 提供标准化的工作环境，而不是让每个 Agent 自己配置。

当前状态：有 WorkspaceStrategy（GitWorktree/In-Place/Sandboxed），但仅是隔离策略。

应该做的：

Environment Engineering Framework

- └ Sandbox Provisioning – 按需创建隔离环境（容器/虚拟机/chroot）
- └ Tool Registry – Agent 可用的工具清单和版本
- └ State Injection – 将上下文注入到 Agent 环境中
- └ Resource Limits – CPU/内存/网络/时间的硬限制
- └ Environment Snapshots – 环境快照，支持回滚和克隆
- └ Telemetry – Agent 在环境中的所有行为可观测

环境工程是“驾驭”的核心——不是控制 Agent，而是塑造 Agent 运行的环境，让正确行为自然发生。

具体行动： – 引入容器化支持（docker/podman API） – 设计 Environment Specification（声明式环境配置） – 支持环境模板（“Python 开发环境”、“前端构建环境”等）

19.5 方向四：Self-Evolution Engine（自进化引擎）

目标：系统从多 Agent 协作历史中自动学习，持续进化。

当前状态：已有贝叶斯信念、ELO 课程、技能蒸馏，但面向单一 Agent。

应该升级的：

Self-Evolution Engine (Multi-Agent)

└─ Collective Learning

└─ Pattern Mining

└─ Strategy Evolution

└─ Capability Mapping

└─ Failure Genealogy

└─ Emergent Behavior Detection

– 从所有 Agent 的执行历史中学习

– 发现跨 Agent 的协作模式

– 进化出最优的 Agent 组合策略

– 动态发现每个 Agent 的真实能力

– 追踪失败模式在 Agent 间的传播

– 发现 Agent 协作中涌现的新行为

这是核心差异化。Qoder CLI 没有学习，没有进化，没有从历史中变聪明。Zero_Nine 有。

具体行动： – 将 IntegrationEngine 从单 Agent 扩展为 Multi-Agent – 增加"协作模式挖掘"：当 Agent A + Agent B 组合时，成功率是否高于单独 – 增加"能力地图"：自动发现每个 Agent 擅长的任务类型 – 增加"进化反馈环"：演化出的策略自动应用到下一次编排

19.6 方向五：Observability & Transparency（可观测性与透明度）

目标：让 Agent 的决策过程、执行路径、失败原因完全可观测。

当前状态：有 events.ndjson 和审计链，但颗粒度和实时性不足。

应该做的：

Observability Stack

└─ Structured Event Bus

└─ Decision Trace

└─ Execution Timeline

└─ Anomaly Detection

└─ Compliance Dashboard

└─ Explainability

– 所有事件结构化，支持实时订阅

– 记录每个决策的原因、依据、替代方案

– 可视化时间线，支持回放

– 自动检测异常行为模式

– 实时合规状态

– 每个决策可解释

(为什么选这个 Agent ? 为什么用这个策略?)

多 Agent 环境下, "黑盒"是不可接受的。用户需要知道：为什么选了 Agent A 而不是 B? 为什么失败了? 系统做了什么决策?

19.7 与 Qoder CLI 的关系：追还是不追？

Qoder 能力	是否需要追	原因
Agent 专业化 (Explore/Plan/Debug)	部分追	可借鉴分类思路，但你的 Agent 是外部的，不是内置的
Hook 系统	追	事件驱动扩展对多 Agent 框架至关重要
MCP 动态调用	部分追	但你的 MCP 应面向 Agent 编排，不是工具调用
IDE 集成	不追	定位是框架/基础设施，不是开发者工具
Context 压缩	不追	这是单 Agent 对话优化，与多 Agent 编排无关
Skills 元数据	追	但 Skills 应描述"Agent 能力"，不是"操作指南"

结论：只追与核心愿景直接相关的能力（Hook 系统、事件驱动、元数据），不追与定位不符的能力（IDE、对话优化）。

19.8 三阶段路线图

Phase 1（现在）

└─ Agent Interface Protocol (AIP) 设计

└─ Agent 身份系统 (Agent ID + 信任评分)

└─ Hook 事件总线 (events.ndjson → 实时事件总线)

Phase 2

└─ Environment Engineering (容器化沙箱)

└─ Multi-Agent 审计链 (agent_id + 跨 Agent 规则)

└─ 能力地图 (动态发现 Agent 能力)

Phase 3

└─ 自进化引擎升级 (集体学习 + 协作模式挖掘)

└─ 可观测性 (决策追踪 + 异常检测)

└─ Agent SDK (Rust/Python/TypeScript)

19.9 总体架构愿景



最终定位：Zero_Nine 不是另一个 Agent，而是 Agent 的基础设施——安全、稳定、自我进化、与任何 Agent 无缝连接的编排框架。